Constructing sufficient commutativity relations for finite acyclic categorical diagrams

Benoît Guillemet, Assia Mahboubi, Matthieu Piquerez

arxiv:2402.14485v4; CC BY4.0

Editorial note. Only comcut correctness is counted. The author explicitly notes that its generated set need not be minimal; that remark and diagram remain preserved. Ordinary category lemmas and structural-duality results are uncounted.

Editorial note. Fixed arxivv4 is CC BY4.0. Complete native author article, original macro input and figures retained. Minted highlighting is mechanically rendered with pure-TeX listings while preserving printed code and original math escapes. No Coq execution or certification claimed.

Editorial note. The separately obtained official SWH-anchor code archive has no explicit redistribution license and is excluded from the delivery dependencies. This counted mathematical result has its full author construction and pen-and-paper proof in the CC article; no external proof pointer is substituted for it.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- Finite acyclic quivers, paths, bipaths and generated path relations. All exact author definitions and categorical interpretation retained in the native article.
- Reverse topological ordering of a finite DAG. Standard graph ordering fact; author describes the corresponding implementation and all specialized construction steps retained.

1 Introduction

Abstract nonsense, a non derogatory expression attributed to Steenrod, usually refers to the incursion of categorical methods for a proof step deemed both technical and little informative, and therefore often succinctly described. *Diagrams* are typically drawn in this case, so as to guide the intuition of the audience, and help visualize the existence of certain morphisms or objects, identities between composition of morphisms, etc.

Formally, a categorical diagram is a functor $F\colon J \to \mathcal{C}$, with J a small category called the *shape* of the diagram [13]. Diagrams are depicted as directed

multi-graphs, also called *quivers*, whose vertices are decorated with the objects of $\mathcal{C}$ and whose arrows each represent a certain morphism, between the objects respectively decorating its source and its target. A directed path in the diagram is hence associated with a chain of composable arrows and a diagram *commutes* when all directed paths with same source and target lead to equal compositions. Equalities between compositions of morphisms thus correspond to the commutativity of certain sub-diagrams of a larger diagram. Chasing commutative sub-diagrams in a larger diagram provides an elegant alternative to equational reasoning, when the latter becomes overly technical. Diagrams actually play a central role in category theory, for they provide such an efficient way of delivering convincing enough proofs. Some classical textbooks introduce diagrams as early as in their introduction chapter [10], while others devote an entire section to diagrammatic categorical reasoning [16, Section 1.6] [7, Session 17]. The following diagrammatic proof of Lemma 1 provides a toy illustrative example of this technique.

Lemma 1. *Let $\mathcal{C}$ be a category. For any morphism f and g such that $g \circ f \in Hom(\mathcal{C})$, if $g \circ f$ is a monomorphism, then so is f.*

Proof. Consider $f : A \to B$, $g : B \to C$ and $g \circ f : A \rightarrowtail C$ morphisms in a category $\mathcal{C}$. Here is a very detailed diagrammatic proof, taking place in the diagram of Figure 1.

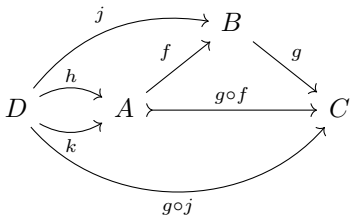


Figure 1: Diagrammatic proof that if $g \circ f$ is a monomorphism, then so is f

We need to prove that for any two other morphisms $h, k : D \rightrightarrows A$ in $Hom(\mathcal{C})$ such that $f \circ h = f \circ k$, we have $h = k$, i.e., that the two-arrows diagram h, k commutes. By hypothesis on h and k, there is an arrow j such that the triangle diagrams respectively formed by arrows h, f, j and k, f, j both commute. By definition of composition, the triangle diagrams formed by arrows $f, g, g \circ f$ and $j, g, g \circ j$ also commute. As a consequence, both triangle diagrams formed by $h, g \circ f, g \circ j$ and $k, g \circ f, g \circ j$ commute. The conclusion follows because $g \circ f$ is a monomorphism. □

Diagram chasing actually refers to a central technique to homological algebra [9, 13], operating on diagrams over abelian categories and used for proving the existence, injectivity, surjectivity of certain morphisms, the exactness of some sequences, etc. The *five lemma* or the *snake lemma* are typical examples of results proved by diagram chasing [9]. However, complex diagram chases (see, e.g., [15, p.338]) only remain readable at the price of hiding non-trivial technical arguments and are, in practice, challenging to rigorously verify by hand. Typically, the reader of a diagrammatic proof is asked to solve instances of variable difficulty of a decision problem hereafter referred to as the *commerge* problem:

Given a collection of sub-diagrams of a larger diagram which commute, must the entire diagram commute? In addition, proofs may resort to non-trivial duality arguments, in which the reader has to believe that a property about diagrams in any Abelian category remains true after reversing all the involved arrows, although the replay of a given proof *mutatis mutandis* cannot be fulfilled in general.

The long-term objective of the present work is thus to build a computer-aided instrument for devising both fluent and reliable categorical diagrammatic reasoning, for 1-categories. The present article describes the implementation of the core of such a tool, as a library for the Coq proof assistant [17]. The design of this library follows two main design principles. First, it aims at being independent from any specific library of formalized category theory or abstract algebra, but rather usable as a helper for any existing one. Second, it should feature enough automation tools for synthesizing the bureaucratic parts of proofs "by abstract nonsense", and formal proofs thereof. The main contributions presented here are thus:

- a deep-embedded first-order language for category theory, geared towards diagrammatic proofs, together with a generic formal definition of categorical diagrams, both based on a previous work by two of the authors [11];
- automation support for proofs by duality in the corresponding reified proof system;
- automation support for the *commerge* problem.

The corresponding code is available at the following url [4]. The rest of the article is organized as follows. Section 2 fixes some vocabulary and describes the corresponding formal definitions. Section 3 describes the deep-embedded formalization of the first-order language, and of the related reified proof system. Section 4 explains the algorithms involved in automating commutativity proofs. We conclude in Section 5 by discussing related work and a few perspectives.

2 Preliminaries

This section fixes some definitions and notations, and introduces their formalized counterpart when relevant. Some of them coincide with the preliminaries of our previous text [11], for the purpose of being self-contained. The description of their formalized counterpart is novel. We do not display implicit arguments in Coq terms. Throughout this article, we use the word *category* for 1-categories.

In all what follows, $\mathbb{N} \coloneqq \{0, 1, \dots\}$ refers to the set of non-negative integers, represented in Coq by the type nat, from its standard library. We also use the standard polymorphic type list for finite sequences, equipped with the library on sequences distributed by the Mathematical Components [12] library. Some names thus slightly differ from those present in the standard library. For instance, the size $|l|$ of a finite sequence l, formalized as `l : list T`, is `size l`, instead of the standard `length l`. We document in comments the definitions we use from this library when their names are not self-explanatory. We recall that `b1 && b2` is the standard notation for the (boolean) conjunction of two boolean values `b1 b2 : bool`. If $k \in \mathbb{N}$, then $[k]$ denotes the finite collection $\{0, \dots, k-1\}$, which is implemented by the sequence `iota 0 n : list nat`.

Definition 2 (General quiver, dual)**.** *A* general quiver $\mathcal{Q}$ *is a quadruple* $(V_\mathcal{Q}, A_\mathcal{Q}, s_\mathcal{Q}\colon A_\mathcal{Q} \to V_\mathcal{Q}, t_\mathcal{Q}\colon A_\mathcal{Q} \to V_\mathcal{Q})$ *where* $V_\mathcal{Q}$ *and* $A_\mathcal{Q}$ *are two sets. The element of* $V_\mathcal{Q}$ *are called the* vertices *of* $\mathcal{Q}$ *and the element of* $A_\mathcal{Q}$ *are called* arrows*. If* $a \in A_\mathcal{Q}$*,* $s_\mathcal{Q}(a)$ *is called the* source *of* a *and* $t_\mathcal{Q}(a)$ *is called its* target*. The* dual *of a quiver* $\mathcal{Q}$ *is the quiver* $\mathcal{Q}^\dagger := (V_\mathcal{Q}, A_\mathcal{Q}, t_\mathcal{Q}, s_\mathcal{Q})$*, which swaps the source and the target maps of* $\mathcal{Q}$*.*

From now on, we casually call *quivers* the special case of quivers with $V_\mathcal{Q}$ a finite subset of $\mathbb{N}$. The formal definition moreover assumes that vertices are labelled in order:

```
(* Data of a quiver *)
Record quiver : Type := quiver_Build {
  quiver_nb_vertex : nat;  (* number of vertices *)
  quiver_arc : list (nat * nat); (* sequence of arrows *) }.

(* Well-formedness condition for quivers :
  all arrows involved in A have a source and target in bound *)
Definition quiver_wf '(quiver_Build n A) : bool :=
  all (fun a => (a.1 < n) && (a.2 < n)) A.

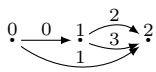
(* The dual quiver of a quiver, with same vertices and reversed arrows *)
Definition quiver_dual '(quiver_Build n A) : quiver :=
  quiver_Build n (map (fun a => (a.2,a.1)) A).
```

An arrow of a formalized quiver q : quiver is thus given by an element in the sequence quiver_arc q, itself a pair of integers giving its source and target respectively. Note that the index in the sequence matters, as the sequence may have duplicate. A formalized quiver is well-formed when the sources and targets of its arrows are in bound.

For the sake of readability, we use drawings to describe some quivers, as for instance:



For a quiver $\mathcal{Q}$ denoted by such a drawing, the convention is that $V_\mathcal{Q} = [\mathrm{card}(V_\mathcal{Q})]$ and $A_\mathcal{Q} = [\mathrm{card}(A_\mathcal{Q})]$, where $\mathrm{card}(A)$ denotes the cardinal of a finite set A. From left to right, the drawn vertices correspond to $0, 1, \ldots, \mathrm{card}(V_\mathcal{Q}) - 1$. Arrows are then numbered by sorting pairs $(s_\mathcal{Q}, t_\mathcal{Q})$ in increasing lexicographical order, as in:



Definition 3 (Morphism, embedding, restriction)**.** *A* morphism of quivers $m\colon \mathcal{Q} \to \mathcal{Q}'$*, is the data of two maps* $m_V\colon V_\mathcal{Q} \to V_{\mathcal{Q}'}$ *and* $m_A\colon A_\mathcal{Q} \to A_{\mathcal{Q}'}$ *such that* $m_V \circ s_\mathcal{Q} = s_{\mathcal{Q}'} \circ m_A$ *and* $m_V \circ t_\mathcal{Q} = t_{\mathcal{Q}'} \circ m_A$*. Such a morphism is called an* embedding of quivers *if moreover both* m_V *and* m_A *are injective. In this case we write* $m\colon \mathcal{Q} \hookrightarrow \mathcal{Q}'$*.*

We also use drawings to denote embeddings. The black part represents the domain of the morphism, the union of black and gray parts represents its

codomain. Here is an example of an embedding of the quiver  into the quiver drawn above.



For the purpose of this work, we actually only need to define formally embedding morphisms, called sub-quivers, which select the relevant vertices and arrows from a quiver:

```
Record subquiver := subquiver_Build {
  subquiver_vertex : list nat; (* labels of the selected vertices *)
  subquiver_arc : list nat; (* indices of the selected arrows *) }.

(* Performs the expected selection of vertices and arrows *)
Definition quiver_restr '(subquiver_Build sV sA) : quiver -> quiver := (...)
```

Here as well, restrictions of quiver only make sense under well-formedness conditions:

```
(* Indices of the arrows to be selected are in bound *)
Definition quiver_restr_A_wf sA '(quiver_Build n A) : bool :=
    all (gtn (size A)) sA.

Definition quiver_restr_V_wf sV '(quiver_Build n A) : bool :=
  uniq sV && (* sV is duplicate-free *)
  all (gtn n) sV && (* all elements of sV are smaller than n *)
  all (fun a => (a.1 \in sV) && (a.2 \in sV)) A. (* any vertex involved in A is
     in sV*)

(* Well-formed condition on the restriction of a quiver *)
Definition quiver_restr_wf '(subquiver_Build sV sA) Q :=
  quiver_restr_A_wf sA Q && quiver_restr_V_wf sV (quiver_restr_A sA Q).
```

Definition 4 (Path-quiver)**.** *The* path-quiver of length k*, denoted by* PQ_k*, is the quiver with* $k+1$ *vertices and* k *arrows* $([k+1], [k], \mathrm{id}, (i \mapsto i+1))$.

A path-quiver can be drawn as:

with at least one vertex. Such a path-quiver is called *nontrivial* if it has at least two vertices.

If $\mathcal{Q}$ is a general quiver, a morphism of the form $p\colon \mathrm{PQ}_k \to \mathcal{Q}$, for some k, is called a *path of* $\mathcal{Q}$ *from* u *to* v *of length* k, where $u \coloneqq p(0)$ and $v \coloneqq p(k)$. A general quiver is *acyclic* if any path of this quiver is an embedding.

If P is a nontrivial path-quiver, we define $\mathsf{st}_P\colon \bullet\bullet \hookrightarrow P$ to be the embedding mapping the first vertex on the leftmost vertex of P and the second vertex on the rightmost vertex of P. Two paths $p_1\colon P_1 \to \mathcal{Q}$, $p_2\colon P_2 \to \mathcal{Q}$ of $\mathcal{Q}$ *have the same extremities* if $p_1 \circ \mathsf{st}_{P_1} = p_2 \circ \mathsf{st}_{P_2}$. We denote by $\mathcal{BP}_\mathcal{Q}$ the set of pairs of paths of $\mathcal{Q}$ having the same extremities. Such a pair is called a *bipath*.

Paths in a formal quivers are defined a ternary relation between two vertices and a sequence of arrows:

```
(* Operations on lists:
  - (_ == _) is a generic boolean comparison test, in this case for lists of
     integers
  - rcons l x  is the list l followed by x
  - unzip[1 | 2] l is the list of fst (resp snd) elements of the list of pairs l
```

```
  - sub p A is the list of elements of A with index in p, in order *)
Definition path (A : seq (nat * nat)) (u : nat) (p : seq nat) (v: nat) : bool :=
  (* all elements in p are in bound *)
  all (gtn (size A)) p &&
  (* p selects in A a list of adjacent arrows from u to v *)
  (u :: unzip2 (sub p A) == rcons (unzip1 (sub p A)) v).
```

A path p from a vertex u to a vertex v can thus be concatenated to a path q from vertex v to a vertex w: when endpoints are obvious, we just write $p{\cdot}q$ the resulting path from u to w. We sometimes abuse notations and write $e{\cdot}p$ and $p{\cdot}e$ when one of the paths contains a single arrow e.

We now introduce a special case of relations on pairs of paths with same extremities in a quiver, called *path relations*. A path relation is an equivalence relation induced from a congruence on the corresponding free category to the quiver. Conversely, in a small category, the composition axiom induces a path relation on the underlying quiver. The formal definition of path relations is actually independent from that of quiver. A path relation is just a family of equivalence relations on sequences of integers, indexed by pairs of integers, that are compatible with the concatenation of paths:

```
Record path_relation := {
  pi_r :> forall u v : nat, relation (seq nat) ;
  pi_equiv : forall u v, equivalence _ (pi_r u v) ;
  pi_cat_stable : forall u v w p p' q q',
      pi_r u v p p' -> pi_r v w q q' -> pi_r u w (p ++ q) (p' ++ q') }.
```

Given a formal quiver and a path relation r : path_relation, (pi_r r u v) is expected to be a relation on the paths from vertex u to vertex v. The full relation on sequences, which relates any two sequences, can be used to complete a partial collection of relations, e.g., the path relation induced by a certain category on its underlying quiver, avoiding this way the need for otherwise cumbersome dependent types.

3 A two-level approach

3.1 Formulas and diagrams

Paraphrasing Mc Lane [10], many properties of category theory can be "unified and simplified by a presentation with diagrams of arrows". Categorical diagrammatic reasoning consists in transforming a proof of category theory into a proof about some quivers, decorated with the data of a certain category. Actually, once the appropriate quivers are drawn, the data themselves can be forgotten, but for the induced path relation, which is the only relevant information for a diagrammatic proof. In [11], we proposed a multi-sorted first-order language for category theory, geared towards diagrammatic reasoning: following the structure of a formula in this language constructs the quivers associated with the corresponding statement of category theory, encoded in the sorts of the variables. We recall its definition:

Definition 5. *We define a many-sorted signature Σ with sorts the collection of finite acyclic quivers. Signature Σ has one function symbol* $\mathrm{restr}_{m\colon \mathcal{Q}'\hookrightarrow\mathcal{Q}}$*, of arity $\mathcal{Q}\to\mathcal{Q}'$, per each* injective *quiver morphism $m\colon \mathcal{Q}'\hookrightarrow\mathcal{Q}$ between two quivers $\mathcal{Q}$ and $\mathcal{Q}'$, and one predicate symbol* $\mathrm{commute}_{\mathcal{Q}}$*, on sort $\mathcal{Q}$, for each finite acyclic quiver $\mathcal{Q}$.*

```
Inductive term :=
    | Var of nat (* variable, named with an integer, $k denotes term (Var k) *)
    | Restr of subquiver & term. (* the 'restr' symbol *)

Inductive formula :=
    | Forall of quiver & formula
    | Exists of quiver & formula
    | Imply of formula & formula (* Denoted with infix symbol -=> *)
    | And of formula & formula
    | FTrue (* Top atom *)
    | Commute of term (* the 'commute' predicate symbol *)
    | EqD of term & term.  (* the equality predicate symbol*)
```

Listing 1: Terms and formulas

Example 6. *Writing the sorts of quantified variables as a subscript of the quantifier, here is for instance a predicate of arity* $\cdot\longrightarrow\cdot \times \cdot\longrightarrow\cdot \times \cdot\longrightarrow\cdot$ *describing composite of arrows:*

$$\mathrm{Comp}(x,y,z): \qquad \exists_{\cdot\rightrightarrows\cdot} w, \quad \mathrm{restr}_{\cdot\rightarrow\cdot\leftarrow\cdot}(w) \approx x \ \wedge\ \mathrm{restr}_{\cdot\rightarrow\cdot\rightarrow\cdot}(w) \approx y$$
$$\wedge\ \mathrm{restr}_{\cdot\rightarrow\cdot\leftarrow\cdot}(w) \approx z \ \wedge\ \mathrm{commute}(w)$$

Example 7. *Here is a predicate of arity* $\cdot\longrightarrow\cdot$ *describing monomorphisms:*

$$\mathrm{Mono}(x): \qquad \forall_{\cdot\rightleftarrows\cdot} w, \quad \mathrm{restr}_{\cdot\rightarrow\cdot\rightarrow\cdot}(w) \approx x \ \Rightarrow\ \mathrm{commute}(\mathrm{restr}_{\cdot\rightleftarrows\cdot\rightarrow\cdot}(w))$$
$$\Rightarrow \mathrm{commute}(\ \mathrm{restr}_{\cdot\rightleftarrows\cdot\rightarrow\cdot}(w))\ \Rightarrow\ \mathrm{commute}(\mathrm{restr}_{\cdot\rightleftarrows\cdot\rightarrow\cdot}(w))$$

Listing 1 is the formalized counterpart of Definition 5. A term t : term is thus either a variable Var n, named with a natural number n, or of the form Restr m t for t a term and m a sub-quiver, seen as a morphism of quivers. Observe that the source quiver of this morphism is left undefined – it only becomes explicit when the term is evaluated. The type formula defines a first-order logic whose atoms stand for equality, commutativity or true. Quantifiers bind de Bruijn indexes and are annotated with a quiver, the sort of the bound variable. To be consistent with the theory Σ, one should only use acyclic quivers. Computing the sort of a term thus requires first annotating each of its variables with a sort. The sort of a term of the form Restr m t is then the quiver obtained by restricting the sort of term t using m.

Given a list l of quivers, providing a sort to each of its variable, a term t : term is well-formed in this context, written term_wf l t, if l is long enough and all the subterms of t have a well-formed sort. In a closed formula, the sort of a variable is read on the corresponding quantifier. More generally, a formula f : formula is well-formed in a context l, written formula_wf l f, if l is long enough to provide a sort to each free variable in f and if all the terms appearing in f have a well formed sort.

Here is the corresponding formal predicate to Example 7.

```
(* We use a notation for easing the definition of quivers
without isolated vertices from their arcs*)
Definition monoQ : quiver := {Q [:: (0,1);(0,1);(0,2);(1,2)]}. (* This is
      ·⇄·→· *)
Definition mapQ : quiver := {Q [:: (0,1)]}. (* This is ·⟶· *)
```

```
(* Lambda_arc constructs a predicate from a sequence of quivers and a formula,
     the first
argument is the arity, providing sorts for the free variables of the second, in
     order.*)
Definition monoF : predicate :=
  Lambda_arc [:: mapQ] (* the one-element arity sequence *)
  (Forall monoQ (
    EqD (Restr {sA [:: 3]} $0) $1
    -=> Commute (Restr {sA [:: 0 ; 2 ; 3]} $0)
    -=> Commute (Restr {sA [:: 1 ; 2 ; 3]} $0)
    -=> Commute (Restr {sA [:: 0 ; 1]} $0))) .
```

The interpretation of a term f : formula as a Coq statement, in sort Prop, is relative to a formal notion of *diagram*, which is by definition an instance of the following structure diagram_type:

```
Record diagram_package (diagram : Type) := diagram_Pack {
  diagram_to_quiver : diagram -> quiver; (* underlying quiver of a diagram *)
  diagram_restr : subquiver -> diagram -> diagram; (* restriction *)
  eqD : equivalence diagram; (* setoid relation on type diagram *)
  eq_comp : diagram -> path_relation; (* path relation *) }.

Structure diagram_type := diagram_type_Build {
  diagram_sort :> Type; (* a diagram_type coerces to its carrier type *)
  diagram_to_package :> diagram_package diagram_sort; }.
```

A diagram is thus a term in the carrier type of an instance of structure diagram_type, which can be seen as a model. A diagram commutes when the associated path relation is *full*, i.e., any two paths in the underlying quiver with same source and target are related:

```
Definition commute (d : diagram_type) (D : diagram) :=
  path_total (diagram_sort D) (eq_comp D).
```

Formal diagrams can be defined from formalized categories, but not only. For instance, one can define an instance of diagram_type with the following carrier type:

```
Record zmod_diagram : Type := ZModDiagram {
  zmob : nat -> zmodType;
  zmmap : forall u v : nat, nat -> {additive zmob u -> zmob v};
  zmdiag_to_quiver : quiver }.
```

where zmodType is a structure for Abelian groups in the Mathematical Components library, and `{{additive A -> B}}` is the type of morphisms between two Abelian groups A and B. We can now explain how to turn a term f : formula into a Coq statement, given a sequence of diagrams:

```
Fixpoint formula_eval (d : diagram_type) (stack : list d) (f : formula) : Prop :=
  match f with
  | Forall Q f => forallD D :: diagram_on Q, formula_eval d (D :: stack) f
  | Exists Q f => existsD D :: diagram_on Q, formula_eval d (D :: stack) f
  | Imply f1 f2 => formula_eval d stack f1 -> formula_eval d stack f2
  | And f1 f2 => formula_eval d stack f1 /\ formula_eval d stack f2
  | FTrue => True
  | Commute t => if term_oeval stack t is Some D then commute D else False
  | EqD t1 t2 =>
    match term_oeval stack t1, term_oeval stack t2 with
    | Some DG1, Some DG2 => eqD DG1 DG2
    | _, _ => False
    end
  end.
```

where the notations forallD D :: diagram_on Q, P and existsD D :: diagram_on Q, P bind variable D in P, so as to quantify P over diagrams D : d with underlying quiver diagram_to_quiver D equal to Q. The evaluation term_oeval stack t : option d of a term t in context stack defaults to None when the context stack is too small and otherwise computes, when possible, the prescribed restriction of the diagrams before the evaluation of the atom using the relevant commutativity and equality predicates.

3.2 Structural duality

As briefly alluded to in the conclusion of our previous article [11], it is possible to prove a duality theorem at the meta-level of the deep-embedded first-order language. The variant presented below is updated to the current, bundled, representation of diagram types, and to a slightly different, albeit equivalent, definition of models. For any formula f : formula, we define its dual formula formula_dual f by structural induction, dualizing all the quivers involved in f. For the sake of readability, the code uses a few notations and coercions:

```
(* We fix a type for diagrams until the end of the section. *)
Variable d : Type.

(* A few local notations to ease reading *)
Local Notation model := diagram_package.
Local Notation model_dual := dual_diagram_Pack.

(* Coercion "conflating" a model (a diagram_type) with carrier type d and its
diagram_package. *)
Local Coercion diagram_type_of := (@diagram_type_Build d).
```

Any model of diagrams M : model d has a dual model_dual M : model d, obtained from M by keeping the same data, but dualizing its quiver and reversing its path relation. We can now prove the property formula_eval_duality, stated in Listing 2, characterizing the evaluation of this dual formula in the dual of a diagram. Note that a coercion is hidden in the type of the argument ctx of formula_eval, to the common carrier type of M and model_dual M.

As a corollary, if a formula holds for all diagrams in a certain diagram type, then so does its dual. The variant duality_theorem_with_theory is equally direct but slightly more interesting, as dependent type P shall be used to describe a specific class of models, e.g. models of a given theory, provided that their description is "auto-dual", cf. Listing 2.

3.3 Proofs

The deep-embedded level also features a data-structure valid_proof for (deep-embedded) proofs of deep-embedded formulas, and implements a checker check_proof for these proofs. A correctness theorem ensures that for any well-formed deep-embedded formula f : formula, a positive answer of the proof checker entails the provability of the interpretation of f in any model d:

```
Theorem check_proof_valid (d : diagram_type) (f : formula) (pf : valid_proof) :
  formula_wf [::] f -> check_proof f pf = true -> formula_eval d [::] f.
```

The deep-embedded level also features a data-structure sequent, used to reify a proof in progress, and a type tactic for actions making progress in a proof:

```
Theorem formula_eval_duality (M : model d) (ctx : seq d) (f : formula) :
  @formula_eval (model_dual M) ctx (formula_dual f) <-> @formula_eval M ctx f.

(* If a formula is valid in every model, then so is the dual formula *)
Corollary duality_theorem (ctx : seq d) (f : formula) :
  (forall M : model d, @formula_eval M ctx f) ->
  forall M : model d, @formula_eval M ctx (formula_dual f).

(* Relativized variant to a specific class P of models *)
Corollary duality_theorem_with_theory (ctx : seq d) (f : formula) (P : model d ->
      Prop) :
  (forall M, P M -> P (model_dual M)) -> (forall M, P M -> @formula_eval M ctx f)
  -> forall M, P M -> @formula_eval M ctx (formula_dual f).
```

Listing 2: Duality theorems

```
Inductive sequent := sequent_Build {
  context : seq quiver;
  premises : seq formula;
  goal : formula; }.

Definition tactic := sequent -> option sequent.
```

Note that there is only one goal attached to a sequent, as disjunction is not part of our formal language. To a sequent with context [:: Q_1 ; ... ; Q_n], premises [:: H_1 ; ... ; H_m] and goal G, one can associate the following term in type formula:

```
Forall Q_1 ( ... ( Forall Q_n (
  Imply   (And H_1 ( ... (And H_m Ftrue) ... ))   G )) ... )
```

The sequent is well-formed if the corresponding formula is a well-formed formula. In the other direction, to any f : formula, one can associate the sequent with goal f and with empty context and no premise. A tactic τ is *valid* when for any well-formed sequent s, if $\tau\, s$ is some s', then s' is also well-formed and the evaluation of s' implies the evaluation of s. In this context, a *valid proof* is just a list of valid tactics.

To check that a valid proof effectively provides a proof of some given formula f, one can perform the following steps. First, compute the sequent associated to the formula. Then, for each tactic in the proof, apply the tactic. If at some point the tactic returns None the proof is not correct. Otherwise, check that the goal of the final sequent is FTrue. If this is the case, then the evaluation of f is true. In the implementation, the verification is performed by the function check_proof, and the conclusion is proven in Theorem check_proof_valid.

The next step is to implement a set of *useful* tactics, and to prove that they are valid. This has been done for basic tactics like introduction and elimination rules, or for more involved tactics like the Rewrite tactic, or the Comauto tactic, for automating the proof of commutative atoms. Starting from the rules of the proof system, more complex tactics combine existing ones in a relevant way, so as to considerably reduce the size of the proofs. In order to be valid, some tactics may also require more assumptions from the model (the type of diagrams) used to evaluate the formulas.

Here is for instance the statement of the formula corresponding to Lemma 1, using an infix notation $-\!\!=\!\!>$ for the Imply constructor of type formula:

```
(* when the quiver of a formula have no isolated vertex, formula_fill_vertices
allows for a shorter description of quivers, only by their arcs. *)
Definition mono_monomPF : formula :=
formula_fill_vertices [::] (
Forall compQ (
  Commute $0 -=> monoF App (Restr {sA [:: 1]} $0) -=> monoF App (Restr {sA [::
    0]} $0))).
```

The reified proof of this statement is currently done by applying successively twelve tactics.[1]

In fact, duality arguments are implemented by instrumenting proofs so as to check that they are amenable to duality arguments, which is more convenient in practice than the structural argument described in Section 3.2. We thus gather a tactic τ and its dual tactic τ^* such that for any sequent s, the dual of τs is equal to τ^* applied to the dual of s. Very often, a tactic and its dual are just identical. A second theorem duality_theorem, not to be confused with that of Section 3.2, ensures that, indeed, if every tactic of a proof of some formula has such a dual tactic, then the proof obtained by taking the dual tactics will be a proof of the dual formula.[2]

```
(* Biproofs are pairs of proofs of same size *)
Structure biproof := biproof_Build {
  biproof_primal : proof;
  biproof_dual : proof;
  biproof_eq_size : size biproof_primal == size biproof_dual; }.

Theorem duality_theorem f (bpf : biproof) :
(* assuming that the tactics in the pair of proofs bpf are pairwise dual *)
  biproofD bpf ->
(* then the primal proof proves a formula iff the dual proof proves its dual *)
  check_proof (formula_dual f) (biproof_dual bpf) =
  check_proof f (biproof_primal bpf).
```

All the tactics currently implemented have a dual tactic (though for some of them, the proof of their validity has not yet been written). A duality argument can be used to prove the dual statement to Lemma 1, and we provide the corresponding deep-embedded formula.[3]

Lemma 8. *Let $\mathcal{C}$ be a category. For any morphism f and g such that $g \circ f \in Hom(\mathcal{C})$, if $g \circ f$ is an epimorphism, then so is g.*

Proof. From Lemma 1, by duality. □

```
Definition epiF :=
  Lambda_arc [:: mapQD]
  (Forall monoQD (
    EqD (Restr {sA [:: 3]} $0) $1
    -=> Commute (Restr {sA [:: 0 ; 2 ; 3]} $0)
    -=> Commute (Restr {sA [:: 1 ; 2 ; 3]} $0)
    -=> Commute (Restr {sA [:: 0 ; 1]} $0))).

Definition epi_mepiPF : formula :=
formula_fill_vertices [::] (
Forall compQD (
  Commute $0 -=> epiF App (Restr {sA [:: 1]} $0) -=> epiF App (Restr {sA [:: 0]}
    $0))).
```

[1] See `mono_monom_pf` in file `diagram_chasing/mono_monom.v`

[2] See file `diagram_chasing/FanL.v`

[3] See file `tests_and_examples/mono_monom.v`

which is similar to term monoPF and formula mono_monomPF except that quivers mapQ, monoQ and compQ have been dualized, respectively into mapQD, monoQD and compQD. The reified proof takes a single tactic, which dualizes the proof of the statement on monomorphisms.

4 Automating commutativity proofs

In order to apply most lemmas obtained by diagram chasing, one first has to prove that a certain diagram is commutative. For instance, the so-called *five lemma*, which allows to prove that some map is an isomorphism, requires to have a commutative diagram over the quiver of Figure 2. Yet proving that such a diagram is commutative by checking one equality per bipaths in this diagram would be excessively tedious. Commutativity of a larger diagram is typically obtained from the commutativity of certain sub-diagrams, and the proof of this implication is often little detailed, or not at all. For instance, in the case of Figure 2, it actually suffices to check four equalities, one for each sub-square of the quiver. More precisely, each pair of paths going from the top-left corner to the bottom-right corner must correspond to equal morphisms. Once these four equalities has been proven, we infer that each square commutes.[4]

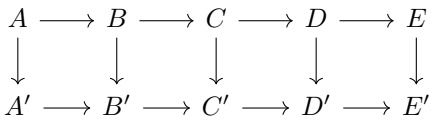


Figure 2: The five-lemma diagram.

In section 4.1, we describe an algorithm for deciding the commerge problem for diagrams with acyclic underlying quivers, so as for instance to automate the proof that the diagram of Figure 2 commutes as soon as the aforementioned four bipaths commute. In section 4.2, we describe a heuristic for discovering a collection of sub-diagrams whose commutativity entails that of a larger one.

4.1 Decision procedure for the commerge problem

In [11], we provided a pen-and-paper proof of the decidability of the commerge problem for diagrams with acyclic underlying quiver, and the undecidability of its generalization to possibly cyclic underlying quivers. In this section, we describe the more practical algorithm implemented by the tactic Comauto. In particular, this implementation comes with a formal proof of correctness, which is the main ingredient in the validity proof of the corresponding tactic.

The algorithm operates on an acyclic quiver $\mathcal{Q}$ and a finite collection $\mathcal{Q}_1, \ldots, \mathcal{Q}_k$ of subquivers of $\mathcal{Q}$, representing the commutativity assumptions. Denote l the union $\bigcup_i \mathcal{BP}_{\mathcal{Q}_i}$, slightly abusing notations by denoting $\mathcal{Q}_i$ the quiver induced by the corresponding subquiver. The algortithm checks whether the smallest path relation $c\ell_{\mathcal{Q}}(l)$ induced by l is full, that is $c\ell_{\mathcal{Q}}(l) = \mathcal{BP}_{\mathcal{Q}}$. Without loss of generality, we can assume that the acyclic quiver $\mathcal{Q}$ is topologically sorted in reverse order, that is, that any path in $\mathcal{Q}$ follows a sequence of

[4]See file `diagram_chasing/Bipath.v` for a formalized proof.

vertices with decreasing labels. The implementation actually performs a topological sort of the quiver [5] and updates the representation of $c\ell_{\mathcal{Q}}(l)$ accordingly.

For any given vertices u and v in $V_{\mathcal{Q}}$, we introduce $\mathcal{A}_{u,v} \subset A_{\mathcal{Q}}$ the set of adjacent arrows of u starting a path to v in $\mathcal{Q}$ and $\bar{\mathcal{A}}_{v,u} \subset A_{\mathcal{Q}}$ that of adjacent arrows of v ending a path from u in $\mathcal{Q}$:

$$\mathcal{A}_{u,v} \triangleq \{e \in A_{\mathcal{Q}} \mid \exists p,\ e{\cdot}p \text{ is a path from } u \text{ to } v\}$$
$$\bar{\mathcal{A}}_{v,u} \triangleq \{e \in A_{\mathcal{Q}} \mid \exists p,\ p{\cdot}e \text{ is a path to } v \text{ from } u\}$$

We define $\mathcal{G}_{u,v}$ the multigraph whose vertices are the elements of $\mathcal{E}_{u,v} \triangleq \mathcal{A}_{u,v} \cup \bar{\mathcal{A}}_{v,u}$. An arc of $\mathcal{G}_{u,v}$ relates vertices $e_1, e_2 \in \mathcal{E}_{u,v}$ when:

- either there is a path in $\mathcal{Q}$ from u to v containing both e_1 and e_2;
- there exists i and p_1, p_2 paths in $\mathcal{Q}_i$ from u to v such that e_1 (resp. e_2) appears in p_1 (resp. p_2).

We moreover introduce the binary relation $R_{u,v}$, on the elements of $\mathcal{E}$: for any $e_1, e_2 \in \mathcal{E}$, $R_{u,v}(e_1, e_2)$ holds if and only if any path p_1 from u to v containing e_1 is related by $c\ell_{\mathcal{Q}}(l)$ to any any path p_2 from u to v containing e_2. If $R_{u,v}$ is full, then $c\ell_{\mathcal{Q}}(l)$ contains all the pairs of paths from u to v.

Lemma 9. *For any vertices $u, v \in V_{\mathcal{Q}}$, if $\mathcal{G}_{u,v}$ is connected, then $R_{u,v}$ is full.*

The decision procedure constructs the multigraphs $\mathcal{G}_{u,v}$ for any pair of vertices, and checks that they are all connected. If so, then $c\ell_{\mathcal{Q}}(l)$ contains $\mathcal{BP}_{\mathcal{Q}}$.

Proof. We prove Lemma 9 for vertices respectively labelled $u + n$ and u, for a vertex $u \in V_{\mathcal{Q}}$, by induction on n. When $n = 0$, then the result holds because paths are empty.

We now assume that the result holds for any u and any $k < n$, and that $\mathcal{G}_{u+n,u}$ is connected. Observe first that as a consequence of the induction hypothesis, and because the graph is topologically sorted in reverse order, any two paths from $u + n$ to u sharing their initial or their final arrow are in $c\ell_{\mathcal{Q}}(l)$. We now fix $u \in V_{\mathcal{Q}}$ and $e_1, e_2 \in \mathcal{E}_{u+n,u}$ and we need to prove that $R_{u+n,u}(e_1, e_2)$. We proceed by induction on the length of a path in $\mathcal{G}_{u+n,u}$ relating e_1 and e_2. If this path is empty, then $e_1 = e_2$ is either an element of $\mathcal{A}_{u,v}$ or of $\bar{\mathcal{A}}_{v,u}$ and we can conclude using the initial observation. We now suppose that there is an arc $e \in \mathcal{E}_{u+n,u}$ and a path t in $\mathcal{G}_{u+n,u}$ such that (e_1, e) is an edge of $\mathcal{G}_{u+n,u}$ and t is a path from e to e_2 in $\mathcal{G}_{u+n,u}$. Let p_1 (resp. p_2) be a path in $\mathcal{Q}$ containing e_1 (resp. e_2). If there is a path p in $\mathcal{Q}$ from u to v containing both e and e_1 then p is related to p_1, by the observation, as the two paths share either their initial or their final arrow. But p is also related to p_2 by the (second) induction hypothesis as p contains e and p_2 contains e_2. The conclusion follows by transitivity of the path relation. Now suppose that e and e_1 respectively belong to q and q_1, paths from $u + n$ to u in some $\mathcal{Q}_i$. Paths q_1 and q are related, by definition of $c\ell_{\mathcal{Q}}(l)$. But q and p_2 are also related by the (second) induction hypothesis, as they respectively contain e and e_2. The conclusion follows by transitivity. □

[5] See `diagram_chasing/TopologicalSort.v`

4.2 Finding sufficient commutativity conditions

In fact, one can even use the computer to guess a sufficient list of equalities entailing that a certain diagram commutes, instead of providing it explicitly by hand. In this section, we explain how to do so in practice. The corresponding algorithm has been implemented under the name *comcut*. Although not strictly needed for the purpose of a tactic, its correctness has been proven formally.[6].

Let $\mathcal{Q}$ be a quiver with vertices $V = [n]$, for some $n \in \mathbb{N}$, and arcs $A_{\mathcal{Q}}$. Moreover, we assume that $\mathcal{Q}$ is topologically sorted in a reversed order, i.e., if an arc goes from u to v, then $u > v$. From such an input, *comcut* returns a list of bipaths $l \subseteq \mathcal{BP}_{\mathcal{Q}}$ such that $c\ell_{\mathcal{Q}}(l)$, that is the smallest path relation in $\mathcal{Q}$ containing l, is equal to $\mathcal{BP}_{\mathcal{Q}}$.

The algorithm works by induction on the size of $\mathcal{Q}$. Let $u_0 := n - 1$ be the top vertex. If there is no arc whose source is u_0, then we just have to apply *comcut* to $\mathcal{Q}$ deprived from the vertex u_0 (note that u_0 cannot be a target). Otherwise, let $a_0 \in A$ be an arc with source u_0. Denote by v_0 the target of a_0. Consider the quiver $\mathcal{Q}'$ obtain from $\mathcal{Q}$ by removing the arc a_0. Applying the algorithm to $\mathcal{Q}'$, we get a list $l' \subset \mathcal{BP}_{\mathcal{Q}'}$ such that $c\ell_{\mathcal{Q}'}(l') = \mathcal{BP}_{\mathcal{Q}'}$. We complete this list to a list l as follows. Let $\widetilde{W} := \mathrm{acc}(u_0) \cap \mathrm{acc}(v_0)$ be the set of vertices accessible both from u_0 and from v_0 in $\mathcal{Q}'$. Set

$$W := \{w \in \widetilde{W} \mid \forall w' \in \widetilde{W} \setminus \{w\}, w \notin \mathrm{acc}(w')\}.$$

For each $w \in W$, select a path p_w from u_0 to w in $\mathcal{Q}'$ and a path q_w from v_0 to w in $\mathcal{Q}'$. Set $l := l' \cup \{(p_w, a_0 \cdot q_w) \mid w \in W\}$.

Proposition 10. *The list l constructed above verifies $c\ell_{\mathcal{Q}}(l) = \mathcal{BP}_{\mathcal{Q}}$.*

Before proving the proposition, let us quickly explain how to get an effective implementation from the above description. To be able to compute accessibility and to reconstruct the different paths, one computes a square matrix indexed by vertices whose (u, v) entry is either empty if there is no nontrivial path from u to v, or contains an arc a such that there is a path from u to v which starts by a. To update such a matrix for the quiver $\mathcal{Q}'$ into a matrix corresponding to the quiver $\mathcal{Q}$, it suffices to set a_0 to all the entries of the form (u_0, v) for $v \in \mathrm{acc}(v_0)$. The rest of the algorithm is easy to write down.

Proof. Let p, q be paths from u to v in $\mathcal{Q}$. Note that the arc a can only appear as the first element of p and q. If a does not appear in p nor in q, or if it appears in both, then (p, q) was already in $\mathcal{BP}_{\mathcal{Q}}(l')$. Hence, up to symmetry, it remains the case where $u = u_0$, p belongs to $\mathcal{Q}'$ and $q = a_0 \cdot q'$ with q' a path of $\mathcal{Q}'$. By definition of $\widetilde{W}$, $v \in \widetilde{W}$. Let $w \in W$ such that v is accessible from w. Let r be a path from w to v (cf. Figure 3). Then, $(p, p_w \cdot r)$ and $(q_w \cdot r, q')$ both belong to $\mathcal{BP}_{\mathcal{Q}'}$, and $(p_w, a_0 \cdot q_w)$ belongs to l. Hence we get the following sequence of relations in $c\ell_{\mathcal{Q}}(l')$.

$$p \ \sim \ p_w \cdot r \ \sim \ a_0 \cdot q_w \cdot r \ \sim \ a_0 \cdot q' = q,$$

which proves the proposition. □

Remark 11. *Note that it may happen that l is not minimal, see Figure 4 for a counterexample.*

[6] See the folder `comcut`

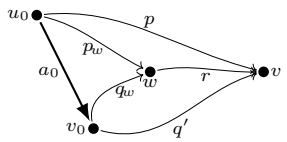


Figure 3: Decomposition of the relation between two paths thanks to an element $w \in W$.

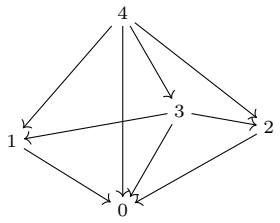


Figure 4: Counterexample to the minimality of the comcut algorithm. Before adding the arc $(4, 3)$, we need four equalities to ensure the commutativity. Adding $(4, 3)$ forces to add two more relations, but one of the previous relations becomes useless.

5 Conclusion

We have described a first step towards the implementation of a generic library for writing reliable categorical diagrammatic proofs, available online [4]. Such a library can serve two purposes. The first one is to assist mathematician authors in writing reliable proofs, the other is to provide the mandatory infrastructure for expanding the existing corpus of formalized category theory, but also of formalized algebraic topology, and homological algebra. As expressed by the Mathlib community [8], the lack for such a tool is major showstopper for the latter. However, the current state of the present library arguably only provides a low level language for categorical statements and the next steps should enrich the collection of formula combinators, e.g. for limits, pullbacks, etc. as well as the gallery of diagram models.

Independence from any library of category theory is achieved by hosting a dedicated proof system inside that of a proof assistant, Coq in this case, following the classic formalization technique of *deep-embedding* [2]. Dependent types allow to formalize a structural duality property for this language. We are not aware of any comparable formal-proof-producing automation tactics for proving the commutativity of diagrams, nor for performing duality arguments. However, some existing libraries of formalized category theory, notably Mathlib, for the Lean proof assistant, and Unimath, a Coq library for univalent mathematics, provide some tools to ease proofs by diagram chasing, either with brute-force rewrite-based tactics, or with a graphical editor for generating proof scripts [6]. Other existing libraries of formalized category theory, including Jacobs and Timany's [18], do not include any specific support for diagrammatic proofs. We refer the interested reader to the later article for a survey of existing libraries

of formalized category theory, which remains quite relevant for the purpose of this discussion, to the notable exception of the more recent Mathlib chapter on category theory. The later serves as a basis for Himmel's formalization of abelian categories in Lean [5], including proofs of the five lemma and of the snake lemma, and proof (semi-)automation tied to this specific formalization. Duality arguments are not addressed. Also in the Mathlib ecosystem, Monbru [14] also discusses automation issues in diagram chases, and provides heuristics for generating them automatically, albeit expressed in a pseudo-language.

Other computer-aided tools exist for diagrammatic categorical reasoning, with a specific emphasis on the graphical interface. Notably, the accomplished Globular proof assistant [1] stems from similar concerns about the reliability of diagrammatic reasoning, but for higher category theory. It is geared towards visualization rather than formal verification and implements various algorithms for constructing and comparing diagrams in higher categories. Barras and Chabassier have designed a graphical interface for diagrammatic proofs which also provides a graphical interface for generating Coq proof scripts of string diagrams, and visualizing Coq goals as diagrams [3]. But up to our knowledge, this tool does not include any specific automation.

References

[1] Krzysztof Bar, Aleks Kissinger, and Jamie Vicary. Globular: an online proof assistant for higher-dimensional rewriting. *Log. Methods Comput. Sci.*, 14(1), 2018. doi:10.23638/LMCS-14(1:8)2018.

[2] Richard J. Boulton, Andrew D. Gordon, Michael J. C. Gordon, John Harrison, John Herbert, and John Van Tassel. Experience with embedding hardware description languages in HOL. In *TPCD*, volume A-10 of *IFIP Transactions*, pages 129–156. North-Holland, 1992.

[3] Luc Chabassier and Bruno Barras. A graphical interface for diagrammatic proofs in proof assistants. Contributed talks in the 29th International Conference on Types for Proofs and Programs (TYPES 2023), 2023. https://types2023.webs.upv.es/TYPES2023.pdf.

[4] Benoît Guillemet, Assia Mahboubi, and Matthieu Piquerez. coq-diagram-chasing. https://gitlab.inria.fr/mpiquere/coq-diagram-chasing, 2024.

[5] Markus Himmel. Diagram chasing in interactive theorem proving. Bachelorarbeit. Karlsruher Institut für Technologie, 2020. https://pp.ipd.kit.edu/uploads/publikationen/himmel20bachelorarbeit.pdf.

[6] Ambroise Lafont. A categorical diagram editor to help formalising commutation proofs. https://amblafont.github.io/graph-editor/index.html, 2024. Short paper presented in the Journées Francophones des Langages Applicatifs.

[7] F. William Lawvere and Stephen H. Schanuel. *Conceptual mathematics. A first introduction to categories.* Cambridge: Cambridge University Press, 2nd ed. edition, 2009.

[8] leanprover-community/mathlib. Condensed mathematics/snake lemma. https://leanprover-community.github.io/archive/stream/267928-condensed-mathematics/topic/snake.20lemma.html, 2021.

[9] Saunders Mac Lane. *Homology*. Class. Math. Berlin: Springer-Verlag, reprint of the 3rd corr. print. 1975 edition, 1995.

[10] Saunders Mac Lane. *Categories for the working mathematician.*, volume 5 of *Grad. Texts Math.* New York, NY: Springer, 2nd ed edition, 1998.

[11] Assia Mahboubi and Matthieu Piquerez. A first order theory of diagram chasing. In *CSL*, volume 288 of *LIPIcs*, pages 38:1–38:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

[12] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, January 2021. doi:10.5281/zenodo.4457887.

[13] J. Peter May. *A concise course in algebraic topology*. Chicago, IL: University of Chicago Press, 1999.

[14] Yannis Monbru. Towards automatic diagram chasing. M1 report. École Normale Supérieure Paris-Saclay, 2022. https://github.com/ymonbru/Diagram-chasing/blob/main/MONBRU_Yannis_Rapport.pdf.

[15] Matthieu Piquerez. *Tropical Hodge theory and applications*. PhD thesis, Institut Polytechnique de Paris, November 2021. URL: https://theses.hal.science/tel-03499730#.

[16] Emily Riehl. *Category Theory in Context*. Dover Publications, 2017. https://math.jhu.edu/~eriehl/context.pdf.

[17] The Coq Development Team. The coq proof assistant, June 2023. doi:10.5281/zenodo.8161141.

[18] Amin Timany and Bart Jacobs. Category theory in coq 8.5. In *FSCD*, volume 52 of *LIPIcs*, pages 30:1–30:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016.